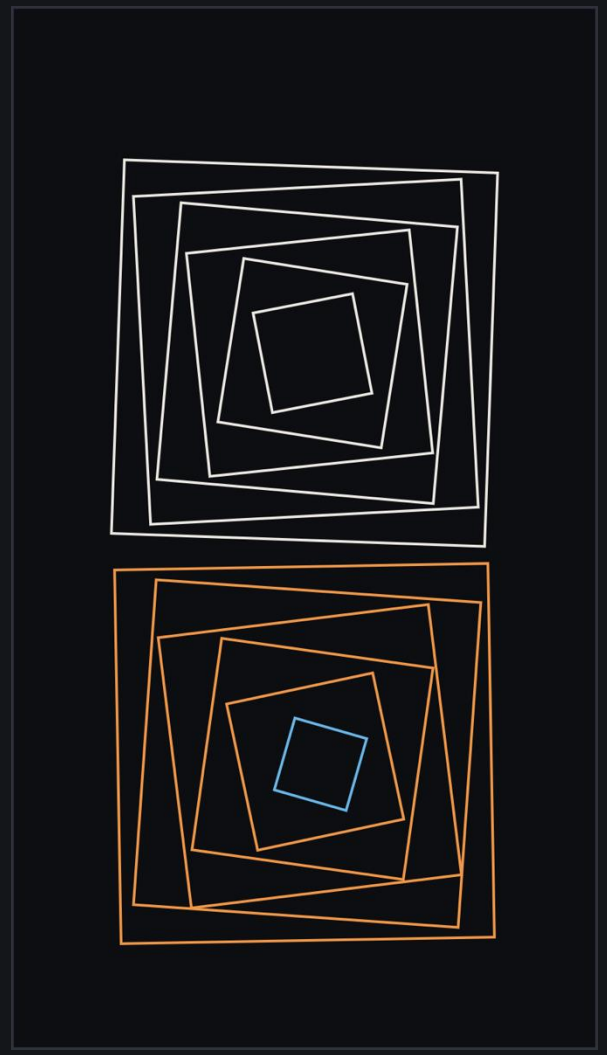
LMC 3402 · GENERATIVE ART · LAB 1

Building Generative Systems with an AI Partner

VS Code · Cline + Gemini · Git · one HTML file that draws for a vertical 4K screen and an A3 pen plotter

Dr. Daniel Phelps · Fall 2026



WHERE WE ARE

From art history to your own system

Lecture

Evolution of Algorithmic Creativity

Galanter, Nees, Molnár, Cage, Reas & Fry, Hobbs

Lecture

Abstract Art & Artistic Process

Spontaneity vs. intention; a personal process

This lab

Lab 1: AI-assisted workspace

VS Code, Cline, Gemini, Git and a first scaffold

Blocks

Build, test, refine

Five blocks, two studio meetings a week

Dec 11

Open studio

Vertical 4K screen + A3 pen plot · 11:20 AM–2:10 PM

Today's question: how do you build a system that makes art, with an AI writing much of the code?

01

The system is the artwork

What generative artists have always done, and what changes when an AI writes the code

> “Generative art refers to any practice where the artist uses a system … which is set into motion with some degree of autonomy contributing to or resulting in a completed work of art.”
>
> — Philip Galanter, *What is Generative Art?* (2003)

This module adds a second autonomous system: **an AI agent that writes the code that runs your rules.**

FROM THE HISTORY LECTURE

Generative artists have always written specs

Vera Molnár

Her “Machine Imaginaire”: she carried out a pre-set program by hand, step by step, before she had a computer, to make “almost exhaustive series, without any omission or repetition.”

For you: write the rules down before anything gets built.

John Cage

In *Not Wanting to Say Anything About Marcel*, I Ching chance operations set the colors, sizes and angles of every element on the Plexiglas layers.

For you: decide what is left to chance, and within what limits.

Tyler Hobbs

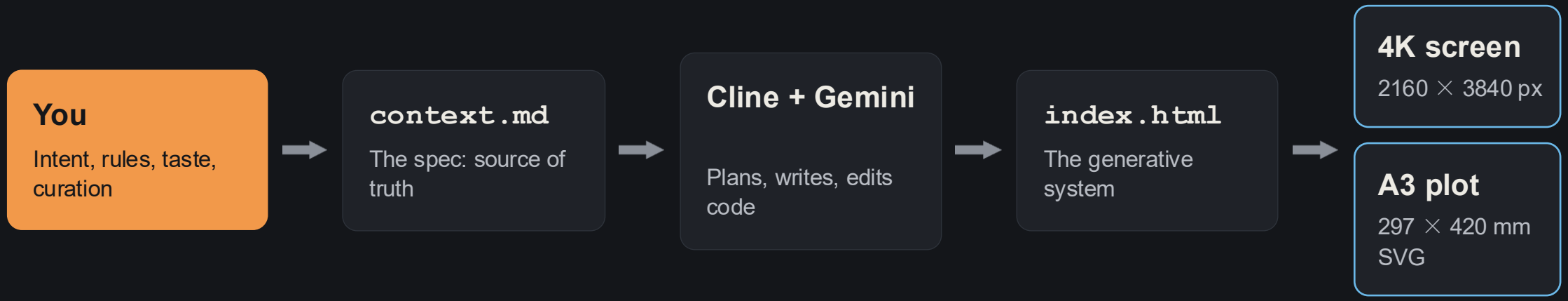
Long-form work like *Fidenza* shows every output of the script: “Nobody, including the collector, the platform, or the artist, knows precisely what will be generated.”

For you: the system has to be good on every seed, not just one.

Sources: Generative Art lecture (LMC 3402); Hobbs, “The Rise of Long-Form Generative Art” (2021)

THE WORKFLOW

Five layers between your idea and the wall



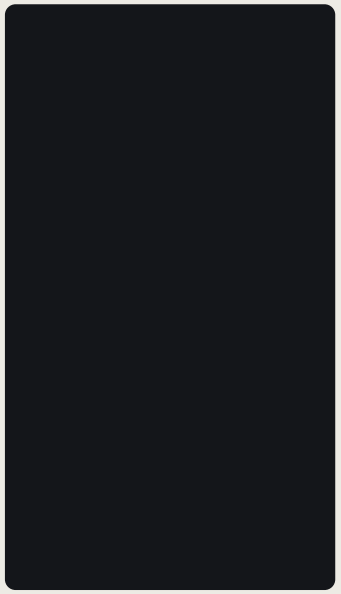
✓ At every hop you review the plan, test in the browser, and commit only what you would sign.

You write the rules.
The agent writes the code.
The code makes the art.

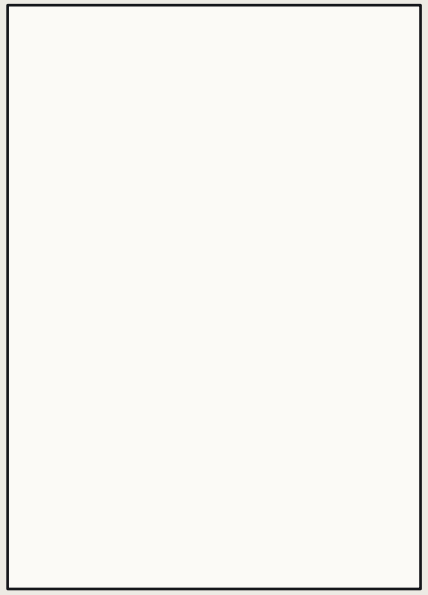
You stay the author: of the intent, the rules, and what gets kept.

THE FINAL PROJECT

One system, two very different outputs



Screen 9:16



A3 1:√2

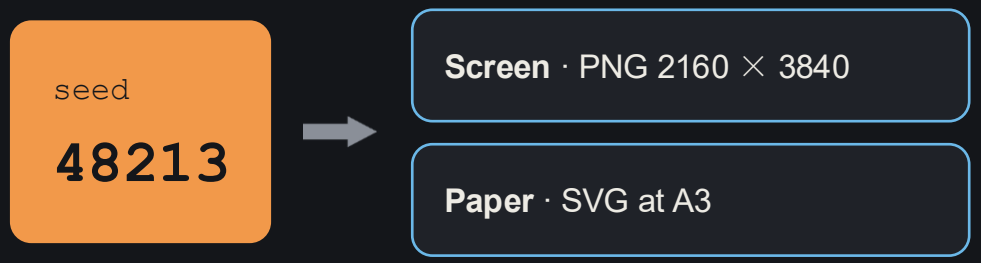
	Vertical 4K screen	A3 pen plot
Size	2160 × 3840 px	297 × 420 mm
Ratio	9 : 16 (0.56)	1 : √2 (0.71)
Medium	Light, color, motion	Ink, strokes only
File	Live canvas / PNG	SVG paths
Seen	From across a room	Up close

So: **compose in 0–1 units, never pixels,** and decide how a piece re-fits: crop, letterbox or re-flow.

The frames are drawn to scale at the same height.

REPRODUCIBILITY

Same seed, same piece



- A seeded random generator replaces Math.random(): one number rebuilds the whole composition.
- Find a seed you like on screen, then plot exactly that piece.
- Keep a Seed Log in context.md. The seed, not a 30 MB PNG, is your archive.

Long-form platforms work the same way: each mint's hash is the seed for one fixed script.

FROM THE ABSTRACT ART LECTURE

Spontaneity vs. intention, in code

Spontaneity

- Seeded randomness and noise
- Emergent behavior you didn’t plan
- An agent’s unexpected suggestion

Intention

- Rules, constraints, parameter ranges
- The spec you write in context.md
- Choosing which seeds to keep

Your context.md opens with an **artist intent**: the “what” and “why” of your process.

02

Working with an AI coding agent

Habits that keep the agent useful and keep you in charge

BEST PRACTICES

Six habits for coding with an agent

1 Source of truth

After each feature, have the agent update context.md: files, milestones, seed log.

2 Small files

Past ~300 lines, ask it to split scripts into separate .js files first. Big files make models stall or truncate.

3 Drawing ≠ output

Rules and geometry live in one place; the screen renderer and SVG exporter only read from it.

4 Spec first

Ask for a plain-text plan with no code. Edit the plan, then say “implement it.”

5 Review before writing

Make the agent explain its changes before it touches any file. Approve each step.

6 Raw error logs

Paste the exact red error from the browser console, not your description of the bug.

Adapted from the course project guide, Part 2

HABIT 4 · SPEC FIRST

Ask for a plan before you ask for code

VAGUE

"Add a flow field."

The agent guesses the structure, the ranges and the export, and writes hundreds of lines you never agreed to.

SPEC-FIRST

"Draft a plain-text Markdown spec for how our flow field stores its parameters (grid size, noise scale, particle count, step length) and how the SVG exporter turns particle paths into polylines at A3. Do not write any code yet."

Read it, edit it, add it to context.md. Then: "Implement the spec."

HABITS 1, 5 AND 6

Three prompts to keep on hand

Before any change	"Explain exactly what you plan to change in our files before modifying anything."
When it breaks	"Analyze this error log and fix the cause: [paste the full red error from the console]"
After it works	"Update context.md: the file list, the milestone checklist, and add seed 48213 to the Seed Log."

Open the console: F12 (Windows) or Cmd + Option + J (Chrome on Mac). Copy the whole error, file name and line number included.

03

Lab 1: Set up your workspace

Six steps from an empty folder to a running, committed generative system

LAB 1 · OBJECTIVE

Build the hub you'll use for the rest of the module

By the end of this lab you have VS Code, an AI agent, Git, and a single **index.html** that draws a seeded composition for a vertical 4K screen and exports it for an A3 plotter.

1
Workspace
Folder, VS Code, git init

2
API key
Google AI Studio

3
Agent
Cline + Gemini in VS Code

4
Spec
Write your context.md

5
Scaffold
First prompt → index.html

6
Commit
First save point + deliverables

No npm, no build tools: everything runs from one HTML file opened in the browser.

STEP 1 OF 6

Optional: Create the workspace

1. Make a folder: **lmc3402-generative-system**
2. VS Code › File › Open Folder… › pick it
3. Open the terminal: Ctrl + ` or View › Terminal
4. Run the commands on the right
5. Create a file named **.gitignore** with the three lines shown

```
# first time on this computer only
git config --global user.name "Your Name"
git config --global user.email "you@gatech.edu"
# every new project
git init
```

```
# .gitignore
.DS_Store
.env
exports/
```

exports/ keeps large PNG and SVG renders out of Git. The seed recreates them.

STEP 2 OF 6

Optional: Get a Gemini API key

1. Go to **aistudio.google.com** and sign in with a Google account
2. Click **Get API key**, then **Create API key**
3. Copy the key. You will paste it into Cline in step 3.

Treat it like a password

- Never paste it into index.html, context.md or a commit
- Cline keeps it in VS Code’s own storage, outside your project folder
- Stay on the free tier: don’t turn on billing for this class
- Leaked it? Delete the key in AI Studio and make a new one

The link in the original handout pointed to google.com; use aistudio.google.com.

STEP 3 OF 6

Optional: Connect Cline to Gemini

1. Extensions (Cmd/Ctrl + Shift + X) › search **Cline** › Install
2. Click the Cline icon in the sidebar, then the gear
3. API Provider: **Google Gemini**; paste your key
4. Choose a model from the table

Model	Use it for
gemini-3.8-flash	Default: plans, specs, tricky bugs. Low free daily limit.
gemini-3.5-flash-lite	Routine edits; much higher free quota
gemini-2.5-flash	Named in the older guide; closed to new API users since Sept 18, 2026

An agent sends many requests per task. Check your limits in AI Studio and switch models when you hit one.

Model names checked against Google’s Gemini API docs, Oct 21, 2026.

STEP 4 OF 6 · CREATE CONTEXT.MD IN THE FOLDER

Write your source of truth

```
# Workspace Blueprint
## Project Overview
- Concept: [what does your system make?]
- Artist intent: [the what and the why]
- Outputs: 2160×3840 px screen
+ A3 (297×420 mm) SVG pen plot
- Tech: one index.html, Canvas, plain JS
## System Rules
1. [a rule, e.g. lines never cross]
2. [what is random, in what range]
3. [what never changes]
```

```
## Architecture Rules
1. All randomness from one seeded RNG
2. Compose in 0–1 units, never pixels
3. Drawing logic separate from export
4. Past ~300 lines: split into .js files
## Seed Log
- [seed]: [why it works]
## Milestones
- [ ] M1 Scaffold + PNG/SVG export
- [ ] M2 … (see the milestones slide)
```

Full template: context.md, shared with this deck. Replace every [bracket] with your own idea.

STEP 5 OF 6 · PASTE INTO CLINE EXACTLY

Your first scaffold prompt

```
Read context.md. Build a single self-contained index.html (no build tools, no frameworks, no CDN links) for our generative art system:

1. A canvas at 2160 × 3840 px (vertical 4K), scaled with CSS to fit the window without distortion.
2. A requestAnimationFrame loop targeting 60 FPS, with a small HUD showing the FPS and the current seed.
3. A seeded random number generator, so the same seed always draws the same image.
4. A placeholder system: lines whose positions depend only on the seed and on 0–1 coordinates.
5. Keys: R = new seed, S = save a full-size PNG, P = export the same composition as an A3 portrait SVG (297 × 420 mm), strokes only.

Explain your plan before creating any file.
```

Approve each step Cline proposes. Then open index.html in your browser: double-click it, or drag it into a Chrome window.

Prompt text is also in the shared files, ready to copy.

STEP 5 · TEST BEFORE YOU COMMIT

Check the result in the browser

- ✓ The tall canvas fits the window without stretching
- ✓ The HUD shows about 60 FPS and a seed number
- ✓ R draws a new composition and the seed changes
- ✓ S saves a PNG that is 2160 × 3840 px
- ✓ P saves an SVG that opens at 297 × 420 mm, lines only, no fills
- ✓ No red errors in the browser console

Something fails? Paste the console error into Cline (habit 6). Still broken after two tries? **Roll back** and re-prompt.

Check image size: Finder › Get Info (Mac) or Properties › Details (Windows).

STEP 6 OF 6

Commit your baseline and submit

1. Source Control (Cmd/Ctrl + Shift + G)
2. Hover **Changes**, click **+**
3. Message, then ✓ **Commit**:

```
feat: scaffold generative system with 4K canvas,
seeded RNG and PNG/SVG export
```

Deliverables · due [date]

1. Screenshot: index.html running in the browser, HUD visible
2. Screenshot: VS Code file tree with index.html, your filled-in context.md and .gitignore
3. Screenshot: git log --oneline showing at least one commit
4. One PNG and its matching SVG from the same seed, the seed in both file names

Submit on Canvas. Deliverable 4 is new: it proves both outputs of the final work from day one.

SUGGESTED BACKLOG FOR CONTEXT.MD

Five milestones to the open studio

Milestone	Goal	Evidence
M1 Scaffold	Canvas, 60 FPS loop, seeded RNG, PNG + SVG export	Lab 1 deliverables
M2 Rules	Replace the placeholder with your own rules and parameters	Spec in context.md, a growing seed log
M3 Screen	Tune color, motion and scale for 2160 × 3840 seen from across a room	Test on the vertical display
M4 Paper	Stroke-only SVG at A3; control line count and pen travel	A first test plot
M5 Final	Curate seeds; finish both outputs	Open studio · Dec 11 · 11:20 AM–2:10 PM

Map each milestone to one of the five blocks, and tick it off in context.md as you go.

Replaces the game milestones in the original guide (entities, collision, difficulty).